Open-end shock-tube change-detection analysis

Detect and characterise any change in high-speed-camera recordings of the open end of a shock tube — the moments when flow comes out of the tube and something happens outside it: a plume, a density gradient, ejected particles, a faint shimmer.

A recording is a directory of thousands–to–millions of single-frame TIFFs from a high-speed camera (Photron FASTCAM or Phantom). Each frame is a 2-D grayscale image. For most of the recording the scene is quiescent — a static background of the tube exit and its surroundings. At some point the flow exits and the image changes. Unlike the sister project `shock_tube_image_analysis` — which looks for a specific signature (a thin moving shock line) — this pipeline assumes no shape. It subtracts the quiescent background and flags any spatially-coherent departure from it, however faint.

The pipeline has two stages:

1. **`detect_activity.py`** — streams through every frame and finds the stretches where the scene changes (the "events"). It also builds the recording's x–t diagram and searches it for moving fronts (a shock), including fronts too faint to show in any single frame.
2. **`analyze_activity.py`** — reloads each detected event and measures what the change is doing: strength, location, footprint, centroid motion and speed, growth/decay, polarity (brighter or darker than the background). Each event gets a label with its likely cause (shock candidate, flash, camera artefact, particle, noise, …), and an overview sheet answers "is anything seen in this recording?" at a glance.

`full_detect_analysis.py` runs both stages in one command with a single set of options. All three scripts accept either one recording folder or a folder of recordings (batch mode, see Batch processing).

A shared module, **`change_common.py`**, holds the image I/O, the background/noise modelling, the change-detection signal processing, and the camera-metadata parsing (Photron `.cihx`, Phantom `.chd`) used by both. (The `.cihx` parsing is shared verbatim with the sister project, so a recording's camera metadata is read the same way by both pipelines.) **`xt_diagram.py`** builds the x–t diagram and runs the shock search; **`event_overview.py`** labels the events and draws the overview sheet.

Table of contents

- Quick start
- How it works (the core idea)
- Is anything seen? The shock search and the overview
- Event labels
- Dashboards
- Installation / requirements
- Camera metadata (`.cihx`, `.chd`) auto-detection
- Script 1 — `detect_activity.py`
- Script 2 — `analyze_activity.py`
- Both at once — `full_detect_analysis.py`
- Batch processing
- Micro-tube detection — `tube.py`
- Shared modules
- Tuning guide
- Troubleshooting
- Validation

Quick start

```
# Detect + analyse one recording. fps is auto-read from the .cihx or .chd sidecar.
python3 full_detect_analysis.py RUN
```

```
# ...or every recording in a campaign folder, laid horizontally.
python3 full_detect_analysis.py shockTube_Marseille --rotate 90
```

Everything about a recording goes into one folder next to it, **`RUN_analysis/`**, and every file in it starts with the recording's name, so it still says where it comes from once attached to an email:

```
RUN/                              the frames (never modified)
RUN_analysis/
├── RUN_dashboard.html            ← start here: interactive, self-contained
├── RUN_overview.png              the same verdict as one picture
├── RUN_events_summary.csv        labelled events, one row each
├── RUN_events.csv                detected events (detection)
├── RUN_shocks.csv                moving fronts found by the shock search
├── RUN_timeline.png              x–t diagram + activity, whole recording
├── RUN_activity.npz              per-frame arrays and the x–t diagram
├── RUN_moments.csv               strongest moments below the event thresholds
├── RUN_tube.png / RUN_tube.json  the micro-tube found (or why none)
├── RUN_tube_shocks.csv           fronts found inside the tube's bore
├── tube_fronts/                  one sheet per front in the tube
├── moments/                      one picture per strongest moment
└── event_0/                      one folder per event
    ├── RUN_event_0_peak.png      the frame at the peak, change outlined
    ├── RUN_event_0_frames.csv    per-frame measurements
    ├── RUN_event_0_montage.png   difference images across the event
    ├── RUN_event_0_trajectory.png
    ├── RUN_event_0.mp4           movie (skipped, with a warning, without ffmpeg)
    └── frames/                   one treated image per frame
```

The two stages can still be run separately; they share the same folder:

```
python3 detect_activity.py RUN      # -> RUN_analysis/RUN_events.csv, ...
python3 analyze_activity.py RUN     # reads it, adds the rest of RUN_analysis/
```

To get speeds in m/s (and reach in mm), add a pixel size if the recording is not spatially calibrated in PFV (the common case):

```
python3 full_detect_analysis.py RUN --px-size-um 50
```

If the recording is quiescent throughout, `RUN_events.csv` contains only its header — the correct result for "nothing happened".

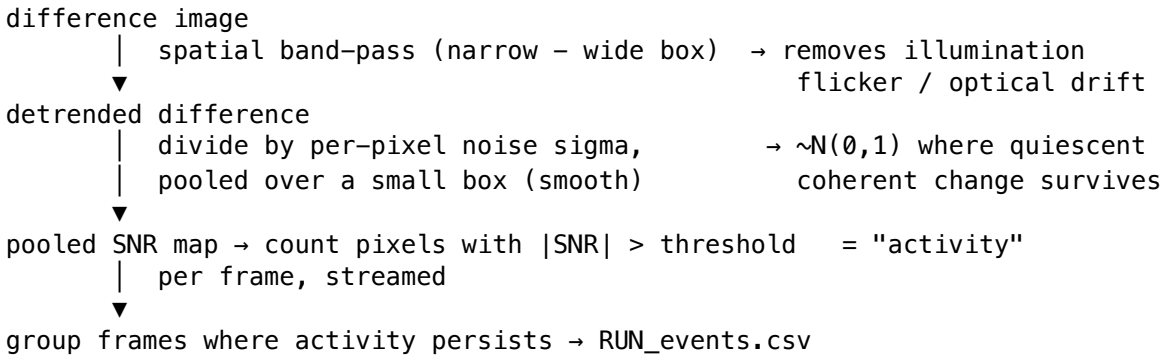
How it works (the core idea)

The whole method rests on one distinction. A pixel at frame t can differ from the static scene for two reasons:

1. A genuine, spatially-coherent change — the flow. Many neighbouring pixels move together, even if each only slightly.
2. Independent per-pixel sensor noise — each pixel wanders on its own.

A single pixel of noise is indistinguishable from a single pixel of faint flow. What separates them is spatial coherence — the spatial analogue of the temporal coherence the sister project uses to recognise a shock. We exploit it in four steps:

```
raw frame (H x W)
       │  subtract per-pixel background          → removes the static scene
       ▼
```



Two nuisance signals are removed along the way, exactly as in the sister project but in 2-D:

- The fixed spatial pattern (tube walls, optics, illumination) → removed by subtracting a per-pixel background, estimated robustly (median) from the quiescent frames.
- A slow illumination drift / global flicker → removed by the spatial band-pass (subtract a wide local-mean of the difference image). A global brightness wobble is coherent across the whole frame and would otherwise swamp the pooled SNR; a localised plume is small compared to the wide box and survives.

The hard part — telling one frame of faint flow from one frame of noise — is answered by demanding coherence on two axes at once:

- Spatial: an event frame must contain a contiguous region of enough active pixels (`--min-area`), not stray noise specks.
- Temporal: the change must persist over several consecutive frames (`--min-len`), not flicker for a single frame.

This is what keeps a quiescent recording from producing false positives.

Robust to flow of unknown duration

The background and the per-pixel noise are estimated from a uniform temporal sample of frames (so memory is bounded regardless of recording length), then refined: the sample frames that turn out to be active are dropped and the model is rebuilt from the rest. So even if the flow lasts a large fraction of the recording, as long as enough genuinely-quiescent frames exist somewhere, they define the background — the flow is never absorbed into it. You do not need to know in advance how long the ejection lasts.

Self-calibrating threshold

Real camera noise is not perfectly independent (row noise, residual fixed pattern), so the pooled SNR is not exactly N(0,1). The model measures the actual pooled-SNR spread on the quiescent frames (the inflation factor, ideally ~1) and scales the pixel threshold by it, so the per-pixel false-positive rate is well-defined whatever the camera's quirks.

Unlit pixels are not measured

Parts of the frame receive no light: outside the window, behind a mask or a wall. Their values are a few dark counts, quantised in coarse steps, and the whole dark area flickers together from frame to frame. The per-pixel Gaussian noise model does not describe that, so these pixels would trigger "events" all the time (on the Marseille recordings, most events came from the unlit strip at the end of the frame). Pixels whose background is below `--dark-frac` (10%) of the median background are therefore excluded, together with an `--edge-margin` band around them — the steep lit/unlit boundary is as unreliable as the frame border. The log reports the measured fraction of the frame. `--dark-frac 0` measures every pixel again.

Flagged frames

Some frames are not a picture of the flow and are flagged:

- camera — the camera changed its frame timing or exposure. A Phantom camera does this at the trigger: the gap before the trigger image is longer and its exposure ~12% longer, so the whole frame jumps in brightness. Read from the per-image timestamps and exposures of the `.chd` header.
- brightness jump — the mean level of the frame departs from its neighbours (running median of 21 frames) by more than 8 robust σ and 0.5%: a flash, a light-source glitch, the sensor recovering after saturation.

Flagged frames are left out of the shock search, marked in grey on the x–t diagrams, and counted in each event (`camera_frames`, `flash_frames`). A run of flagged frames that no event covers becomes an event of its own, whatever its length or area — the trigger jump lasts only 1–2 frames, below `--min-len`, but it is always worth seeing. It is analysed like any event and labelled `camera` or `flash`.

Strongest moments below the thresholds

A quiet recording gives no event, which is the right answer but shows nothing. Detection therefore also lists the `--moments` (8) most active frames outside every event, at least 10 frames apart — what the detector came closest to reporting — in `RUN_moments.csv` (ranked by active-pixel area, then by peak SNR). The overview shows each one as a grey tile with the change outlined and circled. Pure noise also produces a few such moments, so they are for looking, not proof that something happened.

Is anything seen? The shock search and the overview

Counting active pixels answers “did something change?”, but not “was it a shock?”. Two tools answer the second question.

The x–t diagram. For every frame, the pooled SNR map is averaged over the tube height, column by column, and the profiles are stacked over time — the classic shock-tube x–t diagram (x along the tube horizontally, frame number downwards). The tube axis must be horizontal in the (rotated) frame: use `--rotate`. In it:

- a front spanning the tube height — a shock — keeps its full strength under the height average and draws a straight slanted line; the slope is its speed;
- a small object (dust) is diluted by the average and leaves short specks;
- a change of illumination hits every x at once: a horizontal band.

The shock search. A faint front may be invisible in any single frame, but it adds up along its track. The search sums the (unit-noise) x–t diagram along straight tracks for every speed between `--shock-vmin` (4 px/frame) and `--shock-vmax` (a quarter of the frame width per frame, i.e. seen in at least 4 frames), in both directions, over as many frames as the front stays in view (4 to 32). Over K frames the noise grows as $\sqrt{K}$ but a real front grows as K, so the score (in σ) of a real front keeps rising while noise stays near 5–6. Tracks scoring at least `--shock-k` (7) are reported in `RUN_shocks.csv`, one per front. To stay robust, the diagram is clipped at ±5 σ (a flash cannot dominate a sum), flagged frames are zeroed, and a 33-frame running mean is removed so a slow, stationary change does not pass for a slow front.

Sensitivity. The search also computes the faintest front it would have reported, by passing a synthetic full-height line through the same processing: the log and the overview give it as a fraction of the image brightness, e.g. “a front of ≥ 0.70% of the brightness seen over 8 frames would have been reported”. When nothing is found, that is the meaningful statement: nothing brighter than that crossed the field. On the Marseille recordings it is 0.6–0.9% over 8 frames, 0.3–0.5% over 32 frames.

The overview sheet, `RUN_analysis/RUN_overview.png`, puts it together:

1. the verdict — shock candidates or none, with the sensitivity; flagged frames (trigger, camera, brightness jumps); the events by label;
2. the x–t diagram with candidates bracketed by yellow guides, next to the activity curve with events coloured by label;
3. a zoomed x–t diagram around each of the best candidates, where the slope of a fast front is visible (at full-recording scale it looks flat);

4. one tile per event (strongest evidence first, up to 24): the frame at its peak with the active region outlined, titled with its label;
5. one grey tile per strongest moment below the thresholds (see above), also marked on the activity curve.

Dashboards

The analysis also writes HTML dashboards: single files with every image and number embedded (no movies), which open in any browser, offline, and can be moved or e-mailed on their own.

Shot dashboard — **`RUN_analysis/RUN_dashboard.html`**

- the verdict in tiles: shock search, sensitivity, events by label, flagged frames, strongest moment below the thresholds;
- Explore the recording: the x–t diagram next to the active-area and mean-brightness curves, all on the same frame axis. Hover for the values (frame, time from the trigger, position, change in σ); drag vertically to zoom on a range of frames, double-click to reset; event marks, flagged frames, shock candidates and strongest moments are drawn on it, and clicking one opens it;
- Events: a table you can sort and filter by label; clicking an event shows its frames, duration, size, the reason for its label, its peak frame, montage and trajectory pictures and its active-area curve, and zooms the diagrams on it;
- the strongest moments below the thresholds, the shock search results with the sensitivity table, the camera information (from `.cihx` / `.chd`), image quality (brightness, pixel noise) and the analysis settings used; the overview and timeline figures at the bottom.

Campaign dashboard — **`CAMPAIGN/CAMPAIGN_dashboard.html`**

- tiles: shots, shock candidates, events by label, sensitivity range;
- data checks, found automatically: the same recording exported twice (frames compared by fingerprint across all shots), a camera header listing more images than were exported, missing frame rate, several frames with irregular timing, shots not analysed yet or analysed with an older version, settings that differ between shots;
- the shots table (sortable) with an x–t thumbnail, frame rate, exposure, trigger, events, shock-search result, sensitivity, noise, brightness, and a link to each shot dashboard (the links need the `_analysis` folders next to the campaign file);
- across shots: sensitivity, pixel noise, brightness, best shock-search score and events by label compared as bar charts, and every shot's active area and brightness against the time from its trigger, on a common axis.

They are written at the end of every analysis (`--no-dashboard` to skip): the shot page after each shot, the campaign page after a batch run (or after a single shot: every campaign page in the folders above it is refreshed). To rebuild them from existing results without re-running anything:

```
python3 dashboard.py shockTube_Marseille            # campaign + every shot
python3 dashboard.py shockTube_Marseille/26284_1_9  # one shot
```

Shots analysed before these additions (no `NAME_activity.npz`) get a reduced page and are flagged in the data checks; re-run them with `--force` for the full dashboard.

Event labels

Each event in `RUN_events_summary.csv` gets a `label` and a `label_reason` with the numbers behind it. The first rule that matches wins:

Label	Rule	Typical cause
`shock-candidate`	overlaps a shock-search candidate	a moving front: look at it first
`slow-drift`	lasts ≥ 20% of the recording (and ≥ 100 frames)	illumination or background drifting

Label	Rule	Typical cause
`camera`	contains a camera-flagged frame and lasts ≤ 4 frames	trigger frame (timing/exposure change)
`flash`	contains a brightness-jump frame	flash, light glitch, sensor recovery
`global`	at its peak, covers ≥ 20% of the measured field	illumination change
`stripe`	largest blob ≥ 60% of the tube height tall and ≤ 15% of it wide	a front caught in 1–2 frames, or a sensor-row artefact: check by eye
`vibration`	a rigid sub-pixel image shift explains ≥ 50% of the change (R^2)	setup or camera vibrating
`particle`	largest blob holds ≥ 50% of the active pixels and is ≤ half the tube height in both directions	dust, droplet, particle
`noise`	anything else	scattered pixels near the threshold

The features used are also written per event: `peak_area_frac`, `blob_w_px`, `blob_h_px`, `blob_frac`, `shift_r2`, `shift_px`, `camera_frames`, `flash_frames`, `shock_candidates`. The rules are simple on purpose; adjust them in `event_overview.classify` if your recordings call for it.

Installation / requirements

- Python 3.9+
- NumPy
- Pillow (PIL) — TIFF reading
- Matplotlib — for the plots, timeline, treated frames and movies (all on by default)
- ffmpeg — only for the MP4 movies; must be on PATH. Without it (or if it is on PATH but fails to start) the movies are skipped with a warning and everything else runs. Not a pip package: `brew install ffmpeg` (macOS) or `sudo apt install ffmpeg` (Debian/Ubuntu).

No SciPy / OpenCV / tifffile required. Install the Python deps with:

```
python3 -m pip install -r requirements.txt
```

The detector reads frames one at a time during its streaming pass and keeps only the fixed-size background maps plus a bounded temporal sample in memory, so it scales to very long recordings. Memory is `O(--bg-sample)` frames, not `O(total frames)`.

Data layout

```
RUN/
├── RUN.cihx            # Photron camera-metadata sidecar (optional but recommended)
├── RUN000001.tif       # frames, named so alphabetical order == temporal order
├── RUN000002.tif
└── ...
```

Frame files must sort alphabetically into temporal order (zero-padded numbers do this automatically). The integer frame number is taken from the last run of digits in each filename. Frames may be any 2-D size; RGB frames are collapsed to luminance.

Several recordings can sit side by side in a campaign folder, each processed in turn when the campaign folder is given (see Batch processing); each gets its analysis folder next to it:

```
shockTube_Marseille/
├── 26284_1_5/              # frames of one shot
├── 26284_1_5_analysis/     # everything about it
```

```
├── 26284_1_7/
├── 26284_1_7_analysis/
└── ...
```

Camera metadata (**`.cihx`**, **`.chd`**) auto-detection

If a Photron `.cihx` (or `.cih`) file, or else a Phantom `.chd` file, is present in the recording directory, both scripts read it automatically — you do not need to pass camera parameters on the command line.

Photron **`.cihx`**

Field read from `.cihx`	Used for
`recordRate` (fps)	event duration in µs, speed in px/s and m/s
`resolution`	reported in the log
`colorInfo/bit` (depth)	reported in the log
`shutterSpeedNsec`	exposure time, reported
`deviceName`	reported in the log
`plugin/calibration`	spatial pixel size (m/s, mm output)

Frame rate (**`fps`**) is always applied — it is reliable. Spatial pixel size is only applied when the recording is genuinely calibrated. The PFV default (`scaleMode = 0`, `sizeOfPixel = 1.0`, `magnification = 1.0`) is treated as uncalibrated: the scripts deliberately refuse to invent a physical scale from that placeholder, log a note, and report motion in px/frame and px/s only. To get m/s either calibrate the scale in PFV, or pass `--px-size-um`.

Phantom **`.chd`**

When Phantom Camera Control (PCC) exports a recording to TIFFs, it saves the header of the original `.cine` file next to them as `<first image>.chd` (e.g. `Img000000tif.chd`). It is a binary Vision Research cine header:

Part	What it holds	Used for
file header	first image number, image count, trigger time	trigger position in the exported files
bitmap header	width, height, bits per pixel	reported
setup	frame rate, exposure (ns), real bit depth, camera model, serial, UUID	fps; reported
tagged block 1002	one timestamp per image ($1/2^{32}$ s)	irregular frame spacing
tagged block 1003	one exposure per image	irregular exposure
tagged block 1007	one time code per image	—

The trigger image (image number 0) sits at file index `-(first image number)`. Frames whose spacing differs from the rest by more than 5%, or whose exposure differs by more than 2%, are flagged as camera frames (see Flagged frames). There is no spatial calibration in a cine header: pass `--px-size-um` for m/s. If the header describes more images than were exported, the log says so and the files are assumed to start at the first image.

Command-line `--fps` / `--px-size-um` always override the sidecar.

Script 1 — **`detect_activity.py`**

Purpose: scan a whole recording and output one row per stretch where the scene changes.

Usage

```
python3 detect_activity.py INPUT_DIR [options]

# Simplest: fps auto-read from the .cihx or .chd -> RUN_analysis/RUN_events.csv, ...
python3 detect_activity.py RUN

# Restrict to a region of interest
python3 detect_activity.py RUN --row-lo 40 --row-hi 360

# More sensitive (catch fainter flow), with a larger pooling window
python3 detect_activity.py RUN --pix-k 5 --smooth 7 --min-area 12
```

What it does

1. Lists frames in temporal order.
2. Builds the quiescent-scene model — a per-pixel background and per-pixel noise — from a uniform sample of `--bg-sample` frames, refined to exclude any active frames (`change_common.build_background`).
3. Streams through every frame, forms the band-passed, noise-normalised pooled SNR map, and reduces it to a per-frame activity score: the number of pixels whose pooled SNR exceeds the (inflation-scaled) `--pix-k`.
4. Marks frames whose active-pixel area reaches `--min-area` as active, and groups consecutive active frames into events — bridging gaps up to `--max-gap`, keeping those of length within `[--min-len, --max-len]`.
5. Flags camera and brightness-jump frames (see Flagged frames); runs of them outside any event become events of their own.
6. Lists the strongest moments outside the events in `NAME_moments.csv`.
7. Builds the x–t diagram from the same pass and runs the shock search (see Is anything seen?).

Options

Option	Default	Meaning
`input_dir`	—	Directory of single-frame TIFFs, or a folder of such directories (positional; see Batch processing).
`--pattern`	`*.tif`	Glob selecting frame files.
`--out-dir`	`<input_dir>_analysis`	Folder receiving every output (single shot only).
`--force`	off	Batch mode: redo shots whose `NAME_events.csv` already exists.
Region of interest		
`--row-lo` / `--row-hi`	all	Restrict the analysed rows.
`--col-lo` / `--col-hi`	all	Restrict the analysed columns.
`--edge-margin`	6	Pixels ignored at each border (unreliable optics edge). Capped at 1/16 of the frame's smaller side, so a thin strip (e.g. 512×16 px cropped around the tube) keeps its rows.
`--rotate`	`0`	Rotate frames clockwise by 0/90/180/270° after cropping (e.g. `90` to lay a vertical tube horizontally). Crop bounds stay in original-image coordinates; detection is unaffected, only output orientation changes.
Background / noise model		
`--bg-sample`	150	Frames sampled uniformly to build the background.

Option	Default	Meaning
`--noise-floor-frac`	`0.25`	Floor on per-pixel σ (fraction of the median σ).
`--no-refine-bg`	off	Skip dropping active frames from the background.
`--refine-k`	`4.0`	Robustness of the active-frame cut during refinement.
`--dark-frac`	`0.1`	Exclude unlit pixels: background below this fraction of the median background (plus an `--edge-margin` band). `0` measures every pixel.
Change detection		
`--smooth`	5	Side of the spatial box pooling the SNR map. Larger = more sensitive to faint, broad change.
`--detrend-band`	101	Wide box subtracted from the difference image (band-pass) to remove flicker/drift. Make it ≫ the change size; `0` disables.
`--pix-k`	`6.0`	Per-pixel pooled-SNR threshold (σ). Lower (≈5) for more sensitivity; raise to reject noise.
`--min-area`	auto	Active-pixel count for a frame to be “active”. Auto = `max(8, frac·H·W)`.
`--min-area-frac`	5e-5	Sets the auto `--min-area`.
Event grouping		
`--min-len`	3	Min consecutive active frames per event (temporal persistence).
`--max-len`	none	Max frame span of an event (unlimited by default — flow can last).
`--max-gap`	2	Max inactive frames bridged within an event.
Shock search		
`--no-shock-search`	—	Skip the x–t shock search.
`--shock-k`	`7.0`	Report tracks scoring at least this (σ). Pure noise stays below ~6.3 on the test recordings.
`--shock-vmin`	4	Slowest front searched (px/frame). Slower tracks pick up stationary changes.
`--shock-vmax`	width/4	Fastest front searched (px/frame); the default means seen in ≥ 4 frames.
`--xt-bin`	4	Columns merged in the x–t diagram (px), and px per bin along the tube’s bore diagram.
`--tube`	`auto`	`auto`: look for a micro-tube and, if there is one, also search for fronts inside its bore (see Micro-tube detection); `off`: skip.
Strongest moments		
`--moments`	8	How many strongest moments below the event thresholds to list in `NAME_moments.csv` and on the overview; `0` disables.
Metadata / diagnostics		
`--fps`	from `.cihx`/`.chd`	Camera frame rate; overrides the sidecar.
`--save-activity` / `--no-save-activity`	on	Write `NAME_activity.npz`: per-frame arrays and the x–t diagram, read by the analysis overview.
`--plot-timeline` / `--no-plot-timeline`	on	Write `NAME_timeline.png`: the x–t diagram and activity for the whole movie.

All files go into `--out-dir`, prefixed with the shot name `NAME`: `NAME_events.csv`, `NAME_shocks.csv`,

`NAME_timeline.png`, `NAME_activity.npz`.

Output: **`NAME_events.csv`**

One row per detected event:

Column	Description
`event_id`	0-based index, referenced by Script 2.
`start_frame`, `end_frame`, `peak_frame`	Frame numbers of the first / last / most-active frame.
`n_active`	Number of active frames in the event.
`span`	`end_index − start_index + 1` (including bridged gaps).
`peak_area_px`, `mean_area_px`	Active-pixel area at the peak / on average.
`peak_snr`	Strongest pooled SNR in the event.
`peak_energy`	Largest integrated
`polarity`	`brighter` or `darker` than the background.
`onset_cx`, `onset_cy`, `peak_cx`, `peak_cy`	Change centroid (px) at onset / peak.
`camera_frames`, `flash_frames`	Flagged frames within the event (±1 frame).
`shock_candidates`	Ids of shock-search candidates overlapping the event in time.
`start_index`, `end_index`, `peak_index`	Internal 0-based frame ordinals (for debugging).

If no events are found the file contains just the header row.

Output: **`NAME_shocks.csv`**

One row per moving front found by the shock search (header only if none):

Column	Description
`candidate_id`	0-based index, referenced by `shock_candidates` in the events CSV.
`score`	Track score in σ (≥ `--shock-k`).
`start_frame`, `end_frame`, `n_frames`	Frames spanned by the track.
`x0_px`, `x_end_px`	Position along the tube at the first / last frame.
`v_px_per_frame`, `v_px_per_s`, `v_m_per_s`	Speed (signed: + = towards larger x); px/s needs the fps, m/s the pixel size.
`polarity`	`brighter` or `darker` than the background along the track.
`start_index`, `end_index`	0-based frame ordinals.

Output: **`NAME_moments.csv`**

The strongest moments outside every event (see Strongest moments):

Column	Description
`rank`	0 = strongest.
`frame`, `index`	Frame number and 0-based ordinal.
`area_px`, `peak_snr`	Active-pixel area and strongest pooled SNR in that frame.
`cx`, `cy`	Centroid of the active pixels (px).
`polarity`	`brighter` / `darker` (empty if no active pixel).

Script 2 — **`analyze_activity.py`**

Purpose: take the events found by Script 1 and measure each change precisely.

Usage

```
python3 analyze_activity.py INPUT_DIR [EVENTS_CSV] [options]

# Reads RUN_analysis/RUN_events.csv, writes the rest of RUN_analysis/
python3 analyze_activity.py RUN --px-size-um 50

# Explicit input CSV and output folder, without the movie
python3 analyze_activity.py RUN other/RUN_events.csv --out-dir my_analysis --no-movie
```

What it does

For each event it reloads only that event's frames plus a **`--margin`** of context frames, then:

1. Builds a local background + noise from the margin frames (excluding the core event frames so the change does not bias the baseline), with its own inflation calibration.
2. Measures, per frame: active-pixel area, largest connected blob, change centroid `(x, y)`, bounding box, leading-edge distance from the onset centroid, pooled peak/integrated SNR, and polarity.
3. Summarises the event: onset/peak/end, duration, peak and mean area, peak SNR, maximum reach, and a descriptive centroid-trajectory fit giving the velocity vector `(vx, vy)` and speed. (There is no straight-line acceptance gate — the fit is a summary, not a filter.)
4. Converts to physical units when available: `speed[m/s] = speed[px/frame] × fps × pixel_size[m]`, `duration[µs] = duration[frames] / fps`, reach in mm.
5. Labels the event with its likely cause (see Event labels).
6. Renders two diagnostic PNGs per event (`--plots`), a folder of treated per-frame images (`--dump-frames`) and an MP4 movie (`--movie`). All three are on by default; turn any of them off with `--no-plots`, `--no-dump-frames` or `--no-movie`. The movie is skipped, with a warning, when `ffmpeg` is not installed.
7. Draws `NAME_overview.png` for the whole recording (see Is anything seen?), even when no event was detected — it then shows the shock-search verdict and sensitivity. It reads the `NAME_activity.npz` and `NAME_shocks.csv` written by detection next to the events CSV.

   Keep the detection options (**`--smooth`**, **`--detrend-band`**, **`--pix-k`**, **`--edge-margin`**, **`--row/col-lo/hi`**, **`--rotate`**, **`--dark-frac`**) consistent with what you used in Script 1 so the measurements match — or use `full_detect_analysis.py`, which passes the same values to both.

Options

Option	Default	Meaning
`input_dir`	—	Directory of single-frame TIFFs, or a folder of such directories (positional; see Batch processing).
`events_csv`	`NAME_events.csv` in `--out-dir`	CSV produced by Script 1 (optional positional; single shot only).
`--pattern`	`*.tif`	Glob selecting frame files.
`--out-dir`	`<input_dir>_analysis`	Folder receiving every output, created if needed (single shot only).
`--force`	off	Batch mode: redo shots whose `NAME_events_summary.csv` already exists.
`--row-lo/hi`, `--col-lo/hi`	all	Region of interest (match Script 1).
`--edge-margin`	6	Border pixels ignored (match Script 1).
`--rotate`	0	Clockwise frame rotation (match Script 1).

Option	Default	Meaning
`--smooth`	5	Spatial pooling window (match Script 1).
`--detrend-band`	101	Band-pass width (match Script 1).
`--pix-k`	6.0	Per-pixel SNR threshold (match Script 1).
`--noise-floor-frac`	0.25	Floor on per-pixel σ.
`--dark-frac`	0.1	Unlit pixels excluded (match Script 1).
`--margin`	10	Context frames padded around each event.
`--fps`	from `.cihx`/`.chd`	Camera frame rate; overrides the sidecar.
`--px-size-um`	from `.cihx` if calibrated	Pixel size in µm, for m/s and mm output.
`--plots` / `--no-plots`	on	Write the montage + trajectory diagnostic PNGs per event.
`--dump-frames` / `--no-dump-frames`	on	Write one treated image per frame (raw frame + detected-region outline + centroid) into a per-event subfolder, over the full core+margin window.
`--movie` / `--no-movie`	on	Render an MP4 per event from the same treated frames (skipped with a warning if `ffmpeg` is missing).
`--movie-fps`	12	Playback frame rate of the MP4. (Capture is much faster, so this slows the action down.)
`--overview` / `--no-overview`	on	Write `NAME_overview.png` for the recording.
`--front-sheets` / `--no-front-sheets`	on	Write `tube_fronts/NAME_tube_front_<id>.png` for each front found in a micro-tube (see Fronts inside the tube).
`--dashboard` / `--no-dashboard`	on	Write `NAME_dashboard.html`, and the campaign dashboard after a batch (see Dashboards).

Outputs (in **`--out-dir`**)

All prefixed with the shot name `NAME`; per-event files go into an `event_<id>/` subfolder. Re-running the analysis overwrites the files in place and then removes whatever the previous run left that this one did not write again (events that no longer exist, a movie now switched off, …).

`NAME_dashboard.html` — the interactive, self-contained dashboard (see Dashboards).

`NAME_overview.png` — the recording at a glance: shock-search verdict and sensitivity, flagged frames, x–t diagram, activity coloured by label, zooms on the best candidates, one tile per event.

`NAME_events_summary.csv` — one row per event:

Column	Description
`event_id`	Matches `NAME_events.csv`.
`label`, `label_reason`	Likely cause and the numbers behind it (see Event labels).
`start_frame`, `peak_frame`, `end_frame`	First / most-active / last frame.
`n_frames_active`, `duration_frames`	Active-frame count and total span.
`polarity`	`brighter` / `darker`.
`peak_area_px`, `mean_area_px`	Footprint at the peak / on average.
`peak_snr`	Strongest pooled SNR.
`max_leading_edge_px`	Furthest reach of the active region from the onset centroid.
`onset_cx/cy`, `peak_cx/cy`	Centroid at onset / peak.

Column	Description
`centroid_travel_px`	Straight-line centroid displacement.
`centroid_vx/vy_px_per_frame`, `centroid_speed_px_per_frame`	Centroid velocity vector and speed.
`centroid_speed_px_per_s`, `duration_us`	If fps known.
`centroid_speed_m_per_s`, `max_leading_edge_mm`	If fps and pixel size known.
`peak_area_frac`	Peak active area / measured area.
`blob_w_px`, `blob_h_px`, `blob_frac`	Size of the largest blob at the peak, and its share of the active pixels.
`shift_r2`, `shift_px`	How well a rigid image shift explains the change at the peak, and its size.
`camera_frames`, `flash_frames`, `shock_candidates`	Copied from the events CSV.

`moments/NAME_moment_<rank>.png` — one picture per strongest moment below the thresholds (change outlined, circled).

`event_<id>/NAME_event_<id>_peak.png` — the frame at the event's peak with the active region outlined.

`event_<id>/NAME_event_<id>_frames.csv` — per-frame detail across the analysis window: `frame`, `order_index`, `in_core`, `area_px`, `blob_px`, `centroid_x/y`, `bbox_w/h`, `blob_w/h`, `leading_edge_px`, `peak_snr`, `energy`, `mean_snr`, `polarity`. Margin frames have `in_core = False`.

`event_<id>/NAME_event_<id>_montage.png` — (with `--plots`) a strip of pooled-SNR difference images across the event (red = brighter, blue = darker) with the active region outlined in green and the centroid marked.

`event_<id>/NAME_event_<id>_trajectory.png` — (with `--plots`) the centroid trajectory (coloured by frame) and the active-area-vs-frame growth/decay curve.

`event_<id>/frames/` — (with `--dump-frames`) a folder of one PNG per frame across the full analysis window (detected core plus the ±`--margin` context frames): the raw grayscale camera frame with the detected active region outlined in green and the centroid marked with a red ×, titled with the frame number, a `core`/`margin` tag, the active area and the polarity. Files are zero-padded (`frame_004001206.png`) so they sort into temporal order and play as a flip-book.

`event_<id>/NAME_event_<id>.mp4` — (with `--movie`) the same treated frames rendered as an H.264 movie, one video frame per camera frame, at `--movie-fps` (default 12).

Both at once — `full_detect_analysis.py`

> Purpose: run Script 1 then Script 2 on a recording (or on every recording of a campaign) in one command.

```
python3 full_detect_analysis.py INPUT_DIR [options]
```

```
python3 full_detect_analysis.py shockTube_Marseille/26284_1_5 --rotate 90
python3 full_detect_analysis.py shockTube_Marseille --rotate 90 --no-dump-frames
```

It accepts every option of both scripts. The options they share — region of interest, `--rotate`, `--edge-margin`, `--smooth`, `--detrend-band`, `--pix-k`, `--noise-floor-frac`, `--fps`, `--pattern` — are given once and passed to both stages, so detection and analysis always match, and both write into the same `--out-dir`. Detection-only options (`--min-area`, `--bg-sample`, …) go to Script 1, analysis-only ones (`--margin`, `--px-size-um`, `--no-movie`, …) to Script 2. The outputs are exactly those of the two scripts.

The detection and analysis modules stay independent: this script only imports and chains them.

Batch processing

If `INPUT_DIR` holds no frames itself, all three scripts search every folder below it, at any depth, and treat each one that holds frames as a separate shot, processed one after the other. The search does not go inside a shot folder, nor into `*_analysis` output folders or hidden folders (`.name`). Campaigns can therefore be organised freely, e.g. by day and by test:

```
campaign/
├── campaign_dashboard.html       ← every shot below, named by its path
├── day1/
│   ├── test3/
│   │   ├── T1/                   frames → shot "day1/test3/T1"
│   │   ├── T1_analysis/
│   │   ├── T2/
│   │   └── T2_analysis/
│   └── reference/                frames → shot "day1/reference"
└── day2/
    └── test3/
        └── T1/                   same name as above: no clash, each has
                                  its own T1_analysis/ next to it
```

Each shot gets its own `NAME_analysis/` folder, so `--out-dir` and an explicit `EVENTS_CSV` are refused in batch mode.

Shots already processed are skipped, so a campaign can be re-run after adding new recordings:

Script	Shot skipped when…	Otherwise
`detect_activity.py`	`NAME_events.csv` exists	detect
`analyze_activity.py`	`NAME_events_summary.csv` exists, or `NAME_events.csv` is missing	analyse
`full_detect_analysis.py`	both `NAME_events.csv` and `NAME_events_summary.csv` exist	only `NAME_events.csv` exists → analyse only; neither → detect + analyse

(All in `NAME_analysis/`.)

`--force` reprocesses every shot. A shot that fails (e.g. a corrupt frame) is reported with its traceback and the batch carries on; a summary at the end lists the processed, skipped and failed shots, and the exit code is 1 if any failed. Skipping applies only in batch mode: naming a single recording always reprocesses it. After a batch, the campaign dashboard is rebuilt (skipped shots included).

Micro-tube detection — `tube.py`

Most shots have a straight micro-tube in the field; some are taken in open air (to test the shadowgraph). The tube's position, angle, length and diameter vary from shot to shot. `tube.py` finds it automatically from the shot's background image. In a shadowgraph the tube always looks the same: two thin, straight, parallel dark lines (its walls) with a bright strip between them (its bore).

```
python3 tube.py Donnerstag --rotate 90                # every shot, any depth
python3 tube.py Donnerstag/t400/Test1/26284_1_50 --rotate 90
```

Use the same `--rotate` as for the analysis: the geometry is given in the frame as the analysis sees it. Each shot gets, in its `NAME_analysis/`:

- **`NAME_tube.json`** — the tube's geometry, or why no tube was found. It holds the axis angle, the bore edges, the walls, the visible stretches along the tube, and the bore width in px (`bore_px`, and `bore_px_ends` at both ends of the tube).
- **`NAME_tube.png`** — the background with the bore edges (orange) and the ends of each visible stretch (blue) drawn on it.

A folder of shots also gets **`CAMPAIGN_tubes.png`**, one thumbnail per shot, to check every detection at a glance.

How it decides:

1. Thin dark lines — pixels more than 15 % darker than their lit 9×9 neighbourhood. A broad dark object (rod, holder, unlit area) is not a thin line.
2. Straight lines at every angle (Hough), then pairs of parallel lines 3–40 px apart.
3. The bore must be seen — lit, and brighter than both walls, over at least 60 px in total. A noise-only image never forms such long continuous stretches. A tube hidden in places (e.g. under a holder) is kept as several stretches along the same line.
4. Refinement — the angle (to 0.05°) maximises the contrast of the profile across the tube. Each bore edge is measured in 24-px pieces along the tube and fitted with a straight line of its own, so a tapered bore is followed. Pieces more than 0.5 px off the line are dropped, so a local defect cannot tilt it.

Shots whose image contrast is below 20× the pixel noise (unlit, light off) are reported as unusable rather than searched. On the Marseille and Donnerstag campaigns, every shot is classified correctly (51 of 52 have a lit image).

Fronts inside the tube

Detection (`detect_activity.py` / `full_detect_analysis.py`) does this automatically (`--tube off` to skip). It looks for the tube in the background model it builds anyway, writes `NAME_tube.json` / `NAME_tube.png`, and, when there is a tube, builds a second x–t diagram from the bore's pixels only:

- each bore pixel's change in units of its own noise, `(frame − bg) / σ`, minus the frame's median over the bore (a flash brightens the whole bore at once). There is no spatial smoothing: the bore is only a few px wide, and a pooling box would mix in the walls and the flow outside;
- averaged in `--xt-bin` px bins along the tube (the position `s` runs from the start of its visible part), following its angle and its bore edges; bins where the tube is hidden stay empty.

The same straight-track search as for the whole field runs on it (same `--shock-k`, `--shock-vmin`, `--shock-vmax`). A front confined to the bore keeps its full strength there, whereas the height-averaged diagram dilutes it by the bore's share of the frame height (about 5 of 128 rows here). On Donnerstag, that makes the bore diagram about 6× more sensitive to fronts inside the tube. Noise alone scored at most 5.6 in the bore diagrams of 35 shots (pre-trigger frames), against the report threshold of 7.

Inside or outside? A wave travelling outside the tube (e.g. the blast wave around the source) crosses the bore's rows in the picture too: a shadowgraph integrates along the line of sight, and the wave wraps around the tube. To tell them apart, a third diagram samples a band just outside the walls (2–8 px beyond each outer edge, lit pixels only). For each front found in the bore, that band is summed along the same track:

`where`	Meaning
`inside`	nothing significant outside the walls (score < 2.5): a front inside the tube
`outside too`	significant outside (≥ 3.5) with at least half the bore's amplitude: a wave outside the tube, seen through it
`unclear`	in between, or the band outside is lit on fewer than half the track points measured in the bore

where	Meaning
tube moving	the tube itself moved by ≥ 0.1 px across its axis during the track (flagged frames ignored: a flash fakes a one-frame shift): its walls sweep the bore's pixels, so the "front" may be nothing but the walls moving (overrides the others)

The tube's movement is measured in every frame: a rigid shift d across the axis changes the image around the tube by about $-d \times \partial bg/\partial \rho$, and d is the least-squares fit of that to the change in a band covering the tube and its walls. In quiet frames it scatters by about 0.002 px. In Marseille t10 the tube swings by up to 1 px after the shot; in 1_32 it jumps 0.4 px at the trigger and settles within ~12 frames.

Results:

- **NAME_tube_shocks.csv** — one row per front in the bore: score, where, outside_score and tube_shift_px (largest tube movement during it), frames, start/end along the tube (s0_px, s_end_px) and in the frame (x0_px, y0_px, x_end_px, y_end_px), speed along the tube (negative = towards the tube's start, i.e. towards smaller x for a near-horizontal tube), polarity.
- **tube_fronts/NAME_tube_front_<id>.png** — one sheet per front, written by the analysis, to judge it by eye:
  - the frames along its track (up to 4), background-subtracted (3×3 mean, in pixel-noise units, unlit pixels grey), with every front present at that moment marked: ○ inside, □ outside too, ◇ unclear;
  - the x–t diagrams inside the bore and just outside the walls, side by side, around its frames: a front inside the tube shows only in the first, a wave outside the tube in both;
  - the position along the tube against time of every front in those frames: measured in each frame (hollow marker: flagged frame) and the fitted straight track. With an inside and an outside front together, the title gives their speed ratio;
  - the profile across the tube at the front (at most 8 frames of its track), walls shaded: a front inside the tube changes the bore only.
- Events overlapping a front in the bore get the shock−candidate label ("overlaps front(s) … found in the tube"). NAME_events.csv has a tube_candidates column.
- The timeline and the overview show the bore's x–t diagram next to the whole field's. The overview verdict states the tube, the fronts in it (inside first), or the faintest front the search would have reported.
- The shot dashboard shows the bore's diagram in the explorer, sharing the frame axis and the zoom with the whole field's, plus a "Micro-tube" card with the detection picture, the fronts, their sheets and the sensitivity. The campaign dashboard has "Tube bore" and "Fronts in tube" columns, and a tile counting the shots with a front inside their tube.

Shared modules

change_common.py

Imported by both scripts; not run directly. Key contents:

Frame discovery & I/O - list_frames(input_dir, pattern) — frame paths in temporal order. - find_shots(input_dir, pattern) — the shot folder(s) to process: the folder itself, or every folder below it holding frames, at any depth (batch mode). - shot_paths(shot_dir, out_dir) → ShotPaths — every output path of a shot (folder, prefixed files, per-event folders); output_base(events_csv) — the prefix detection's other files share with the events CSV. - run_shots(tag, shots, batch, process) — run a shot at a time, carrying on after failures in batch mode and printing a summary. - frame_number(path) — integer frame number from a filename. - load_frame(path, row_lo, row_hi, col_lo, col_hi, rotate) — load a frame as a 2-D float image, optionally cropped then rotated clockwise.

Background / noise & change detection (pure NumPy, no SciPy) - box2d_mean(img, k) — k×k box average via a summed-area table, O(H·W). - estimate_bg_noise(stack) — robust per-pixel median background

and MAD noise. - `build_background(files, p)` → `BackgroundModel` — sampled, refined background, noise, measured-pixel mask, and the inflation-scaled pixel threshold. - `valid_mask(bg, edge_margin, dark_frac)` — pixels that are measured: off the border and inside the lit field. - `activity_snr(frame, bg, sigma2, smooth, detrend_band)` — band-passed, noise-normalised pooled SNR map. - `measure_activity(snr, thresh, valid)` → `ActivityMeasurement`. - `largest_blob(mask)` — largest connected component (flood fill). - `robust_threshold(scores, k)` — `median + k·(1.4826·MAD)`. - `level_jumps(level)` — frames whose mean brightness jumps (flash).

Camera metadata - `find_cihx` / `parse_cihx` → `CameraMeta` (Photron; shared verbatim with the sister project). - `find_chd` / `parse_chd` → `CameraMeta` (Phantom), with the trigger index, per-image timestamps and exposures, and the irregular frames. - `resolve_calibration(input_dir, fps_arg, px_size_um_arg, n_files)`.

`xt_diagram.py`

- `xt_row(snr, valid, xbin)` — one row of the x–t diagram.
- `normalise(xt, bad_rows)` — unit-noise, clipped, high-passed diagram.
- `search(z, umin, umax)` — best straight-track score from every (frame, x).
- `candidates(...)` — one reported track per front.
- `sensitivity(model, p, ...)` — faintest front the search would report.
- `draw_xt(ax, ...)` — plot helper.

`tube.py`

- `detect_tube(bg, sigma)` → `(Tube, "")` or `(None, reason)`.
- `Tube` — `angle_deg`, `rho_bore` / `bore_slope` (each bore edge as a line), `rho_wall`, `rho_outer`, `segments` (visible stretches); `coords(shape)` gives the (s, rho) of every pixel, `bore_mask(shape)` the bore pixels where the bore is seen, and `bore_edges(s)` / `bore_at(s)` the edges / width along it.
- `save` / `load` — `NAME_tube.json`.

`dashboard.py` (+ **`dashboard_template.html`**)

- `build_shot(analysis_dir)` / `build_campaign(campaign_dir)` — write the dashboards from the files in the analysis folders only (no frames needed).
- `data_checks(shots, campaign_dir)` — duplicates, incomplete exports, …
- Run on its own to rebuild the pages: `python3 dashboard.py PATH`.
- The page layout, styles and interactive code are in `dashboard_template.html`; the data are embedded into a copy of it.

`event_overview.py`

- `classify(summary, ev, rows, pack, n_total)` — features, `label`, `label_reason`.
- `make_overview(p, summaries, tiles, files)` — `NAME_overview.png`.

Tuning guide

Start from the defaults; they are set so a quiescent recording yields zero events. When you have a recording that does contain flow and detection is off, adjust in this order:

Symptom	Knob	Direction
Faint flow missed entirely	`--pix-k`	lower (e.g. 6 → 5) — more sensitive pixel threshold
Faint, broad flow missed	`--smooth`	raise (e.g. 5 → 7–9) — more spatial pooling
Flow detected but split into pieces	`--max-gap`	raise (e.g. 2 → 4) to bridge dropouts

Symptom	Knob	Direction
Short events rejected	`--min-len`	lower (e.g. 3 → 2)
Too many false positives (noise)	`--pix-k`, `--min-len`, `--min-area`	raise — stricter coherence
Illumination flicker leaking through	`--detrend-band`	keep it ≫ the change size; lower if drift is finer-scale
A diffuse, frame-filling glow is suppressed	`--detrend-band`	raise (or `0`) — it is being band-passed away
Only the tube exit matters	`--row/col-lo/hi`	restrict the ROI to cut spurious triggers elsewhere
Events keep firing at the edge of the lit field	`--dark-frac`, `--edge-margin`	raise — exclude more of the dark / boundary area
A self-luminous event in an unlit area is ignored	`--dark-frac`	**`0`** — measure the dark pixels too (expect more noise events)
Shock search reports noise tracks	`--shock-k`	raise (7 → 8)
A known front is not reported	`--shock-k`, `--shock-vmin/vmax`	lower `--shock-k` (≥ 6.5), check the speed range includes it, check `--rotate` puts the tube axis along x

Use the `RUN_activity.npz` arrays and the `RUN_timeline.png` plot to inspect the activity signal and choose `--pix-k` / `--min-area` deliberately. The detector logs the inflation factor and the effective pix threshold — if inflation is far above 1, the per-pixel noise is strongly correlated and a larger `--smooth` or `--detrend-band` will help.

Troubleshooting

- "no .cihx/.cih/.chd sidecar found" — fine; just pass `--fps` (and optionally `--px-size-um`) yourself.
- "header describes N images but M files were exported" (Phantom) — the per-image flags assume the files start at the first image of the header; check the trigger frame on the overview.
- "recording is NOT spatially calibrated" — expected when the scale wasn't set in PFV. Motion is still reported in px/frame and px/s; add `--px-size-um` for m/s and mm.
- Whole recording flagged as one giant event (labelled `slow-drift`) — part of the scene keeps changing slowly, the background absorbed something it shouldn't, or strong global flicker is leaking through. Look at where the activity sits on the overview; restrict the ROI, raise `--detrend-band`, and make sure enough quiescent frames exist for `--bg-sample`.
- Zero events on a recording you believe has flow — see the tuning table; most often lower `--pix-k` and/or raise `--smooth`. Confirm the flow really is a departure from the background and not within the per-pixel noise.
- **`FileNotFoundError: No frames matching ...`** — check `--pattern` and the directory path. For a campaign folder, the frames must sit directly inside each shot folder (shot folders can be nested at any depth; hidden folders and `*_analysis` folders are not searched).
- "ffmpeg not found or broken, skipping the MP4 movies" — install ffmpeg (see requirements), or pass `--no-movie`. If `ffmpeg -version` fails with `Library not loaded` (a Homebrew library was upgraded under it), run `brew upgrade ffmpeg`.
- A shot is "skipped" in batch mode — its outputs already exist; delete them or pass `--force`.
- Speeds look wrong by a constant factor — check `--px-size-um` and that `--fps` matches the recording (not a default).
- Files named **`… 2.png`**, **`… 2.mp4`**, **`frames 2/`** appear in the analysis folders — iCloud Drive conflict copies, when the data sit in a synced folder (e.g. the Desktop) while outputs are being rewritten. They are not written by the scripts and can be deleted. Keeping the data outside iCloud, or in a folder whose name ends in `.nosync`, avoids them.

Validation

The pipeline ships validated against two cases built from the sister project's real T1 background (`_make_test_data.py` regenerates them):

- True negative: 160 untouched, quiescent T1 frames produce 0 events at the defaults — the background subtraction, band-pass and two-axis coherence gate reject the residual camera noise. (Lowering `--pix-k` to 5 surfaces a single marginal 3-frame noise-tail excursion, which `--pix-k 6` or `--min-len 4` removes — a useful illustration of the sensitivity/robustness trade-off.)
- True positive: a faint moving Gaussian "plume" (peak ≈ 60 DN over ~10 DN noise) injected into the real background, brightening then fading over 14 frames while moving at 45 px/frame, is recovered with the correct extent, polarity (`brighter`), and kinematics — centroid velocity 44.95 px/frame (≈ 388 m/s at 172 800 fps and 50 µm/px), `vy ≈ 0`. The shock search finds it too, as one track at 45.1 px/frame (score 15.7).

The shock search was validated on the real Marseille recordings (Phantom v2012, 100 000 fps, 128 × 608 px, `--rotate 90`):

- Noise level: with no front present, the best track score is 5.4–6.2 on every recording (flagged frames excluded), below the default `--shock-k 7`. Pure Gaussian noise of the same size gives 5.35.
- Injected fronts: a full-height line (Gaussian profile, σ = 2 px) with 1% brightness contrast moving at 40 px/frame, added to frames 300–315 of `26284_1_9`, is reported at −39.6 px/frame (score 9.1; negative because `--rotate 90` reverses the axis) at the right place and time. The same front at 0.5% contrast is not reported (score 6.0), consistent with the computed sensitivity of 0.71% over 8 frames. On the pipeline's own x–t signal, fronts of 0.1 pixel-noise σ at 10 px/frame and 0.2 σ at 40–100 px/frame are recovered.